

We sincerely thank all reviewers for their constructive and detailed feedback. Below, we provide our response, where we have grouped related feedback thematically to address all points in a cohesive manner. We will revise the paper accordingly if given the opportunity.

1. Operational Scenario, Practicality, and Relevance

Target Use Case (Q1 from R-C)

For clarity, we adopt the scenarios defined by Reviewer-C: Scenario 1: Assessing patches for new bugs in a production context. Scenario 2: Assessing patches to benchmark and evaluate APR tools.

Historian, as evaluated in this paper, is primarily designed for Scenario 2.

Fairness of APCA Comparison and Relevance (Q1 from R-A & Q2 from R-C)

Our evaluation protocol is fully aligned with APCA baselines (ODS, CodeBERT, LLM4PatchCorrect, Cache), all of which are also evaluated on benchmark datasets with known ground-truth patch correctness. Though techniques differ, the evaluation setting is identical, ensuring a sound and fair comparison.

Our work is relevant because it directly tackles the patch redundancy identified in our RQ1, a primary driver of wasted manual effort. Historian offers a more efficient paradigm by eliminating the need to relabel recurring solutions.

Historian's Novelty & Technical Contribution (Limited Novelty from R-B)

Historian's novelty is two-fold. First, our work provides a large-scale empirical study (RQ1) that quantifies the dual nature of APR patches: high redundancy (~38% of unique patches are syntactic clones) and high solution diversity (~59% of bugs have multiple distinct fixes).

Second, Historian is the first APCA framework directly engineered from these empirical insights. Its contribution is the paradigm shift from abstract prediction to traceable, evidence-based comparison. Existing tools are predictive methods that use data to train an abstract model. In contrast, Historian introduces a new paradigm where the historical data is the model—a dynamic, concrete knowledge base for direct comparison. This design allows it to: Exploit Redundancy: Automatically validate the large number of recurring patches, directly tackling the primary source of manual labor identified in RQ1. Handle Diversity: Use a multi-reference approach to correctly validate multiple distinct solutions for a single bug, overcoming the single-point-of-failure limitation also revealed in RQ1.

Our technical novelty lies in the framework that operationalizes a principled mapping logic to translate clone detection results into verdicts and a robust, multi-reference voting algorithm to ensure reliability.

Benefits of Adoption (Q1 from R-A)

- **Superior Performance:** The "Punished" setting is a conservative metric for a fair baseline comparison. However, in a realistic semi-automated workflow where Unknown cases are deferred to experts, Historian's performance is superior. Table 6 shows our weighted F1-score with Gemini is 0.91 (Acc: 88.4%), surpassing the SOTA's 0.88 F1 (Acc: 83.8%). It achieves this by trading a small amount of coverage (labeling 41/825 cases Unknown) for significantly higher precision on the patches it can confidently classify.
- **Explainability:** Unlike opaque predictive models, Historian's verdicts are explainable, providing the concrete patch(es) it matched against. The Unknown label also correctly signals when historical evidence is insufficient and expert review is necessary.
- **Adaptability:** Historian's knowledge base can be updated with new validated examples without expensive retraining, making it more adaptable and lower-maintenance than predictive models. Combined with its strong performance using smaller open-source LLMs, it offers an accessible and cost-effective path to SOTA performance.

Practicality and Patch Collection (Q1 & Q2 from R-B)

For our primary use case (Scenario-2), Historian is immediately practical. The "historically validated patches" are the benchmark and community-curated datasets from prior academic studies that researchers already use.

For the extensible use case (Scenario 1 — production/industry use), Historian's core methodology remains applicable, and we view this as a natural direction for future work. This would involve an organization building its own proprietary reference set over time. This can be achieved by: Extracting developer-written fixes from version control to serve as Correct references. Recording the outcomes of manual code reviews for APR-generated patches, where accepted patches become Correct references and rejected ones become Overfitting references.

2. Methodological Rigor and Soundness

Purpose of Cohen's Kappa (R-A's comment on reliability)

In our manual annotation of 4,248 pairs, two authors worked independently, resulting in a Cohen's Kappa of $\kappa = 0.96$. This near-perfect agreement provides objective, statistical evidence that our annotation process was highly consistent rather than being subjective to a single annotator and the resulting labels are a reliable input for our oracle experiment. The few disagreements were resolved by consensus. The formal guidelines and final labels are in our replication package to further support reliability and enable community validation.

Two-Stage Parsing (Q2 from R-A)

Regex-Based extraction uses simple, case-insensitive regular expressions to capture explicit keywords. For binary tasks, the regex is `\b(?:yes|no)\b`, and for clone detection, it is `\b(?:type-1|type-2|type-3|type-4|no)\b`.

In our experiments, the regex successfully parsed ~53% of responses. The zero-shot fallback, used for the remaining 47%, was validated with ~91% accuracy on the regex-parsable set, confirming its reliability. The adherence to the regex format varied across models. A detailed per-model breakdown is available at https://anonymous.4open.science/r/Historian-Artifact/rebutal/model_comparison_zeroshot_table.png. Our best-performing open-source model (Qwen2.5-7B) adhered to the structured format in ~85% of cases, minimizing reliance on the fallback. We will incorporate these details into the paper.

Use of TBar for RQ2/RQ3 (Q3 from R-A & R-C)

The primary driver was the intensive manual annotation effort for the RQ2 oracle study (4,248 pairs), which was infeasible on a larger scale. TBar dataset provided a manageable yet high-quality and representative set for this focused analysis, as its template-based patches are known to be closer to human fixes.

We used the same set for RQ3 to ensure a consistent and direct comparison between the oracle's theoretical performance and the practical, LLM-driven results. This focused analysis on a well-understood patch set allowed for a more controlled evaluation of our various configurations. We will add this rationale and note that studies on more APR tools are key for future work.

Detecting Semantic Equivalence vs. Similarity (Q3 from R-C)

Our ultimate goal is to determine semantic equivalence—that is, whether a candidate patch has the same functional behavior as a known reference patch. To achieve this, we task the LLM with assessing different forms of semantic similarity (such as CC, SS, and SI). We then use our principled mapping logic (Table 2) to determine which degree of similarity is strong enough to infer equivalence. For example, our framework treats Type-1, Type-2, and Type-4 clones as reliable proxies for semantic equivalence, allowing a direct transfer of the correctness label. In contrast, a Type-3 clone is treated as a signal of partial similarity but not equivalence, correctly leading to an Unknown verdict. We will revise the paper to clarify that our framework uses specific, high-confidence similarity signals (e.g., strong clone types) as proxies for semantic equivalence.

Prompt Design Rationale (Q4 from R-C)

Code Clone Detection (CC), Semantic Similarity (SS), and Semantic Identity (SI) were chosen to explore different granularities of semantic relationships. SI is the strictest form. SS is the loosest. CC provides a well-defined framework using the standard Type-1 to Type-4 clone definitions. Our goal was to empirically determine the most effective framing for the LLM, and

our RQ3 results showed the structured CC prompts performed best. We will add this explicit rationale, along with a forward reference to the findings in RQ3 to the revised paper.

Disregarding Unknown Votes in Algorithm 1 (R-C's comment)

Disregarding Unknown votes is a deliberate design choice for achieving principled abstention. An Unknown vote from a pairwise comparison signifies weak or ambiguous evidence. Including these uncertain votes in the final majority count would dilute the strong, confident signals for Correct or Overfitting.

By considering only informative votes, Historian makes a definitive final decision only when there is sufficient evidence. If the evidence is inconclusive (e.g., a tie or only Unknown votes), the system correctly defaults to Unknown, deferring the decision to a human expert. This design allows Historian to intentionally trade full coverage for higher precision and trustworthiness in its verdicts, which we will clarify in the revised paper.

Manual Assessment with Relaxed Conditions (R-A's comment)

"Relaxed conditions" in RQ1 allowed human annotators to identify Type-1/Type-2 clones despite minor syntactic variations that automated clone detection tools might miss. This included trivial changes like the addition or removal of superfluous parentheses. We will clarify this in the revised paper.

Reliability of Historical Labels (R-B's comment)

Our results build upon peer-reviewed, community-standard datasets. Our reference set is sourced from five publicly available, validated patch collections. Additionally, we performed rigorous data sanitization to improve quality. Finally, Historian's majority voting mechanism is inherently resilient to potential residual errors in the labels, as a single incorrect data point is highly likely to be outvoted by the collective evidence. We will state this in our "Threats to Validity" section.

Scope of Experimental Data (R-B's comment)

We acknowledge the limitations regarding the scope of our experimental data. Java offers the richest ecosystem of tools and labeled datasets in the APR community, providing the most impactful context for our initial validation. However, Historian's is language-agnostic, and multi-language validation is a key direction for future work.

Regarding the benchmark scale, while smaller sets like QuixBugs were included for diversity, our findings stem from the aggregated dataset, dominated by larger benchmarks such as Defects4J and Bugs.jar. We will clarify these scoping choices and their implications in the Threats to Validity section.

Dataset and Setup for RQ2/RQ3 (R-A & R-C's comments on clarity)

Our experiment was based on our single-method, deduplicated dataset of 4,635 patches (the sum of 4,128 Correct and 507 Overfitting patches from the "De-duplicated" row of Table 3). We partitioned this set as follows to simulate the assessment of an "unseen" tool:

- Candidate Set: The 139 patches generated by TBar.
- Reference Set: The remaining 4,496 non-TBar patches, which served as our historical knowledge base.

For the analysis, each candidate patch was then paired with every reference patch that targeted the same bug, resulting in the 4,248 pairs for analysis. This setup demonstrates the rigor of our experiment: the 139 candidate patches were not analyzed in isolation but were evaluated against a large and diverse historical set of nearly 4,500 examples. We will revise the paper to explicitly detail this partitioning and pairing methodology for full clarity.

3. Presentation, Artifacts, and Minor Issues

Artifact Availability (R-C's comment)

We have verified and resolved the access issue. The artifact is now accessible at the original link <https://anonymous.4open.science/r/Historian-Artifact/README.md> and mirrored on Zenodo <https://doi.org/10.5281/zenodo.17205776>.

Presentation, Structure, and Minor Issues (R-A & R-C's comments)

We will revise the paper to improve its structure and clarity, and we will correct all noted minor issues. This includes:

- Restructuring the paper: Moving RQ1 to a "Motivating Study" section and relocating the Research Questions.
- Clarifying content: Adding key metrics to the abstract, explaining prompt design as a configuration point, and reordering table discussions.
- Correcting all noted items: Including updates to Tables 4 and 7, fixing the caption in Figure 5, updating the LaTeX header, and correcting all typos.

Once again, we sincerely thank all reviewers for their valuable feedback. We believe their suggestions will greatly contribute to improving the quality of our work.